

**System Requirement Specification /
Dokumen Kebutuhan Perangkat Lunak**

YogaFX Learning Management System (LMS) Web

Dipersiapkan Untuk:

YogaFX

[Alamat Perusahaan/Unit YogaFX]

Dipersiapkan oleh:

[Group ID]

[Study Program / Year / Class]

Dokumen ini menjabarkan kebutuhan fungsional dan non-fungsional dari YogaFX LMS Web, sebuah platform pembelajaran daring berbasis Laravel + Inertia + React yang melayani dua sisi pengguna utama — Student dan Admin — mencakup alur konversi lead menjadi student berbayar, pengelolaan konten pembelajaran bertingkat (tier), asesmen, sertifikasi, serta penagihan installment.

DAFTAR ISI

1 Pembukaan	3
1.1 Tujuan Penulisan Dokumen	3
1.2 Ruang Lingkup Produk/Sistem yang Akan Dibangun	3
2 Deskripsi Umum	4
2.1 Deskripsi Umum Sistem yang Akan Dibangun	4
2.2 Fungsi Utama	4
2.3 Kelompok dan Karakteristik Pengguna	8
2.4 Batasan Desain dan Implementasi	11
3 Kebutuhan Rinci	13
3.1 Kebutuhan Antarmuka	13
3.1.1 Antarmuka Sistem	13
3.1.2 Antarmuka Pengguna	14
3.1.3 Antarmuka Perangkat Keras	14
3.1.4 Antarmuka Komunikasi	14
3.2 Spesifikasi Kebutuhan Fungsional	14
3.2.1 Autentikasi & Kontrol Akses	14
3.2.2 Akuisisi Publik, Checkout & Onboarding	16
3.2.3 Manajemen Konten Pembelajaran & Pengalaman Belajar	17
3.2.4 Asesmen (Assessment)	18
3.2.5 Tugas (Assignment) & Sertifikasi	19
3.2.6 Komersial & Penagihan Installment	20
3.2.7 Manajemen Peserta, Notifikasi Email & Link Control	22
3.3 Kebutuhan Non Fungsional	23
3.3.1 Kebutuhan akan Performansi	23
3.3.2 Kebutuhan akan Keselamatan	23
3.3.3 Kebutuhan akan Keamanan	23
3.3.4 Atribut Kualitas Perangkat Lunak Lainnya	24
3.3.5 Aturan Kebutuhan Operasional	24
4 Kebutuhan Lain dan Model Analisis	25

1 Pembukaan

1.1 Tujuan Penulisan Dokumen

Dokumen ini ditulis untuk mendefinisikan kebutuhan fungsional dan non-fungsional dari sistem **YogaFX LMS Web** secara rinci, sebagai acuan bagi para pengembang sistem (programmer, product owner, dan penguji/QA) dalam membangun, memelihara, dan mengembangkan lebih lanjut sistem tersebut. Dokumen ini juga digunakan sebagai bahan evaluasi Kerja Praktek untuk menunjukkan pemahaman terhadap kebutuhan perangkat lunak dari sistem yang sedang dikerjakan.

1.2 Ruang Lingkup Produk/Sistem yang Akan Dibangun

YogaFX LMS Web adalah platform pembelajaran daring (Learning Management System) milik YogaFX yang digunakan untuk mendistribusikan materi pelatihan yoga secara terstruktur dan berjenjang (tier) kepada peserta (student), sekaligus menjadi kanal konversi calon peserta (lead) menjadi peserta berbayar melalui alur checkout dan onboarding.

Ruang lingkup sistem mencakup:

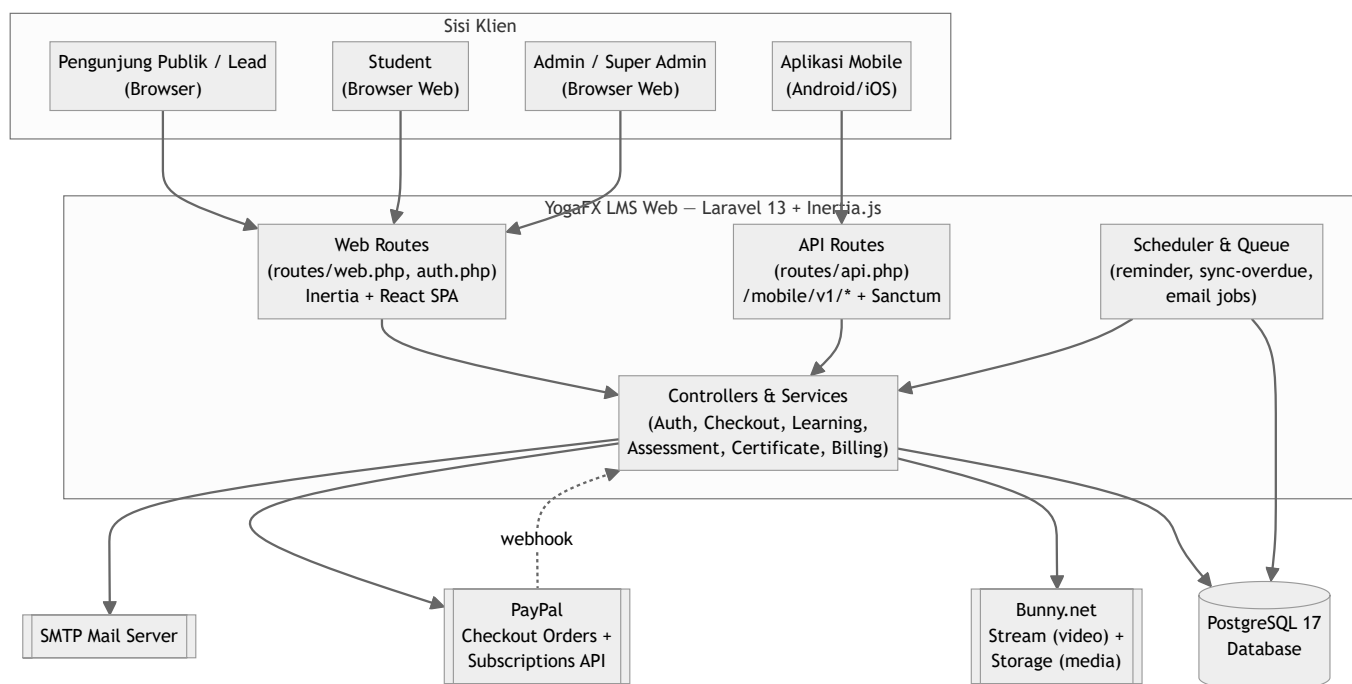
- Halaman publik untuk pendaftaran lead, checkout paket (pembayaran penuh maupun cicilan/installment), serta onboarding peserta baru.
- Sisi **Student**: akses konten pembelajaran (module, lesson, ebook, course/video lecture) sesuai tier keanggotaan, pengerjaan asesmen, pengumpulan tugas (assignment) berupa video, pengunduhan sertifikat, dan pengelolaan profil termasuk upgrade tier.
- Sisi **Admin**: pengelolaan konten pembelajaran, pengelolaan tier & paket komersial, pengelolaan akun student dan admin, peninjauan progres belajar student, penerbitan sertifikat, pengelolaan template notifikasi email, serta pengelolaan tautan unduh aplikasi mobile.
- Backend API mobile (/api/mobile/v1/*) yang melayani aplikasi mobile pendamping dengan fitur setara sisi Student pada web.
- Proses backend terjadwal (cron) untuk pengingat inaktivitas dan sinkronisasi status tagihan installment yang telat.

Sistem ini memberikan manfaat bagi YogaFX berupa satu platform terpadu untuk akuisisi peserta, penjualan paket pelatihan (termasuk skema cicilan dan donasi), penyampaian materi belajar terstruktur, serta penerbitan sertifikasi kelulusan — menggantikan proses manual yang sebelumnya tersebar di berbagai kanal.

2 Deskripsi Umum

2.1 Deskripsi Umum Sistem yang Akan Dibangun

YogaFX LMS Web merupakan produk baru yang dibangun dari nol (bukan migrasi dari sistem lama), menggunakan arsitektur monolith Laravel dengan Inertia.js sebagai jembatan ke frontend React (server-driven SPA), didukung basis data PostgreSQL. Sistem ini terdiri dari empat permukaan utama yang saling terhubung melalui satu basis data dan basis kode yang sama: Web Publik, Web Student, Web Admin, dan API Mobile.



Gambar 2.1 — Diagram Arsitektur Sistem YogaFX LMS Web

Alur tingkat tinggi: pengunjung publik → mengisi lead registration/scoreboard → memilih paket (tier) → checkout (bayar penuh atau cicilan via PayPal) → onboarding (enrollment & signup) → menjadi Student aktif → mengakses konten belajar sesuai tier → mengerjakan asesmen/tugas → menerima sertifikat kelulusan.

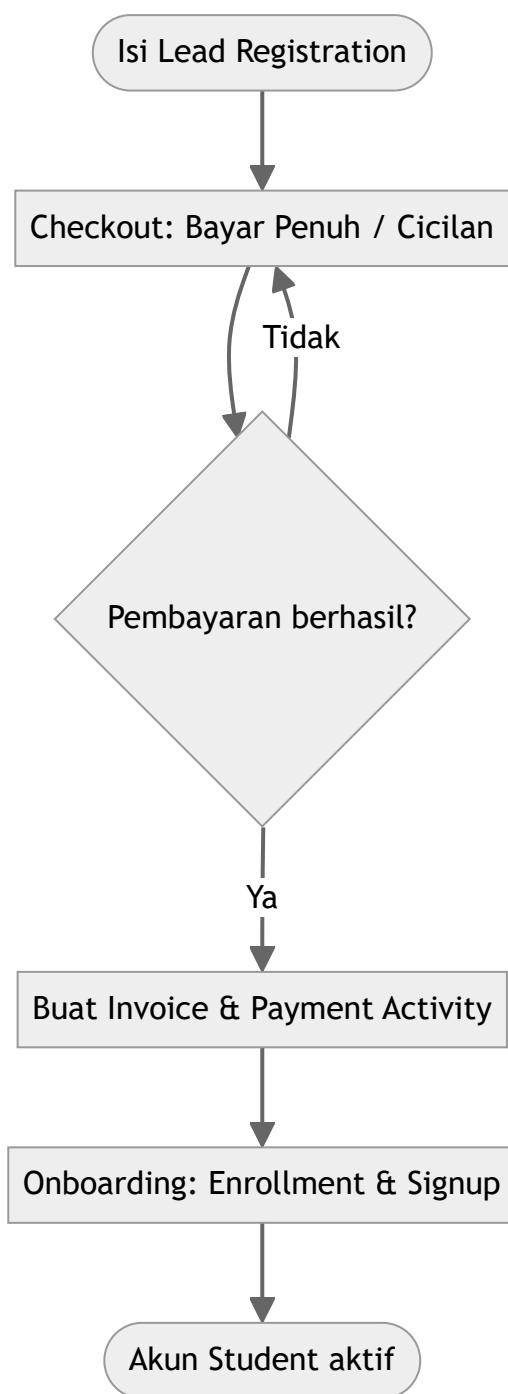
2.2 Fungsi Utama

Fungsi utama sistem meliputi:

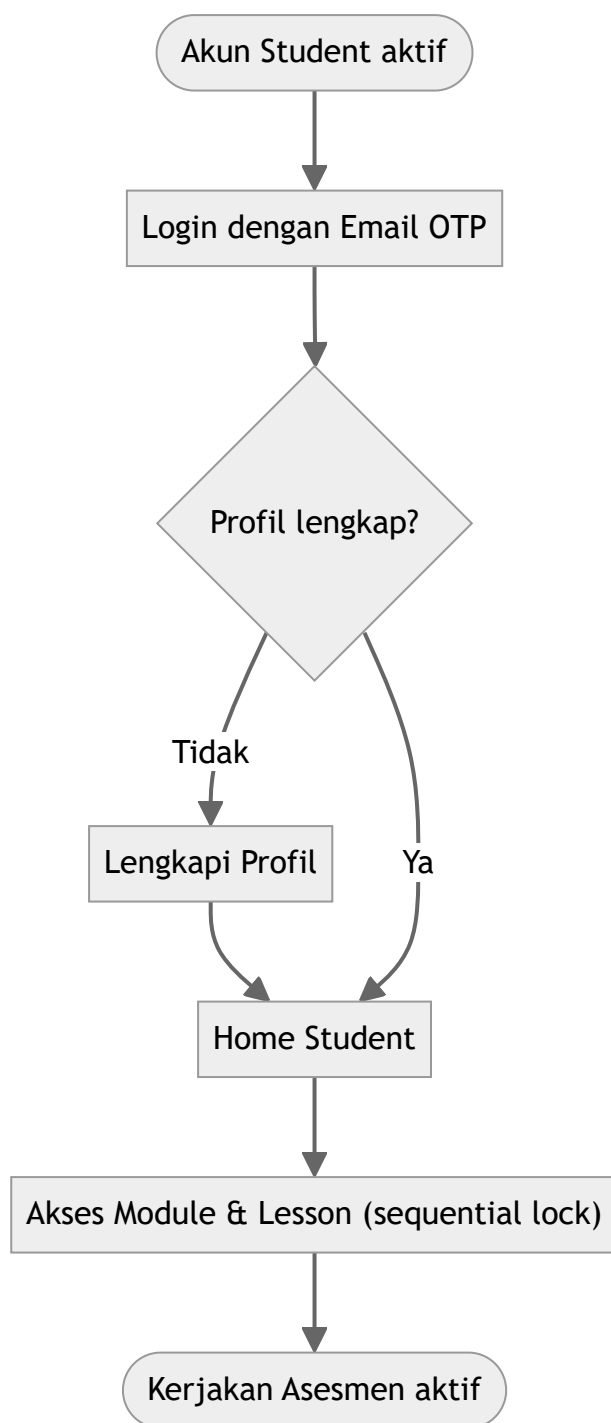
- **Akuisisi & Checkout Publik:** lead registration, landing page per tier/paket, checkout pembayaran penuh maupun cicilan, onboarding enrollment dan signup.
- **Autentikasi & Otorisasi:** login dengan verifikasi OTP email, logout, lupa/reset password, pembatasan akses berbasis role (super_admin, admin, student) dan status aktif akun.
- **Manajemen Konten Pembelajaran (Admin):** CRUD Module, Lesson, Ebook, Course/Video Lecture, Assignment, dan Dialog Content, dengan akses many-to-many terhadap Access Tier.
- **Pengalaman Belajar (Student):** Home, daftar & detail module, detail lesson dengan video (Bunny Stream), progres tonton, ebook preview/download, sequential lesson locking.

- **Asesmen:** builder asesmen (pertanyaan, opsi, jump logic, rentang hasil) oleh Admin; pengerjaan asesmen dan hasil oleh Student.
- **Tugas (Assignment):** pengumpulan video tugas oleh Student, peninjauan & status oleh Admin.
- **Sertifikasi:** pembuatan, pengunduhan, dan pengelolaan sertifikat berbasis kelayakan (tier, penyelesaian modul, persetujuan tugas).
- **Komersial (Package & Access Tier):** pengelolaan paket harga, mata uang, konfigurasi cicilan, dan tier keanggotaan (Starter Kit, Online, Master Class).
- **Penagihan Installment:** kalkulasi jadwal cicilan, integrasi PayPal Subscription, webhook pembayaran, deteksi & pemulihan tunggakan (overdue) secara terjadwal.
- **Manajemen Peserta (Admin):** daftar student, aktif/nonaktifkan akun, reset progres, hapus akun.
- **Notifikasi Email:** template, pengiriman uji, log email, serta pemicu otomatis.
- **Link Control:** pengaturan tautan unduh aplikasi mobile (Google Play/App Store) dan QR code terkait.

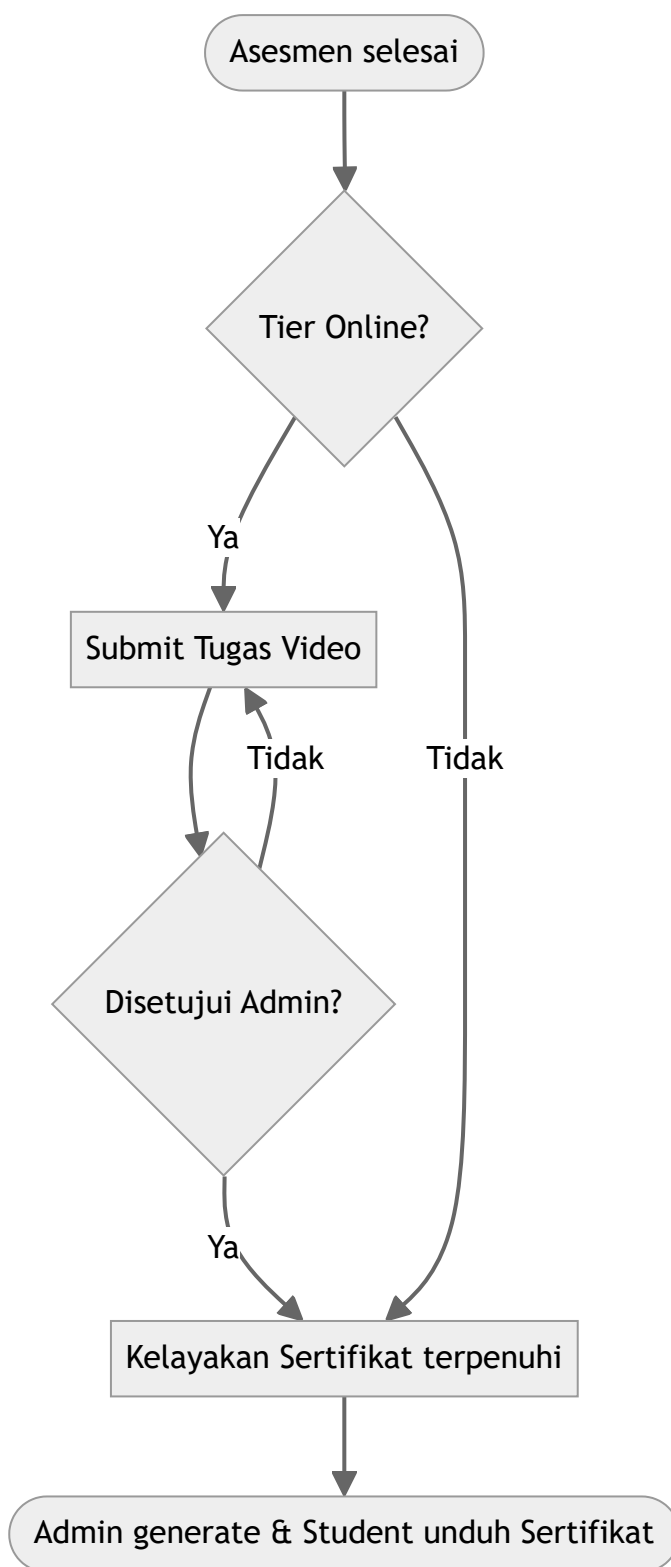
Diagram aktivitas perjalanan utama pengguna dibagi menjadi tiga tahap berurutan agar mudah dibaca: (a) Akuisisi & Checkout, (b) Autentikasi & Akses Belajar, dan (c) Tugas & Sertifikasi.



Gambar 2.2a — Aktivitas Tahap Akuisisi & Checkout



Gambar 2.2b — Aktivitas Tahap Autentikasi & Akses Belajar

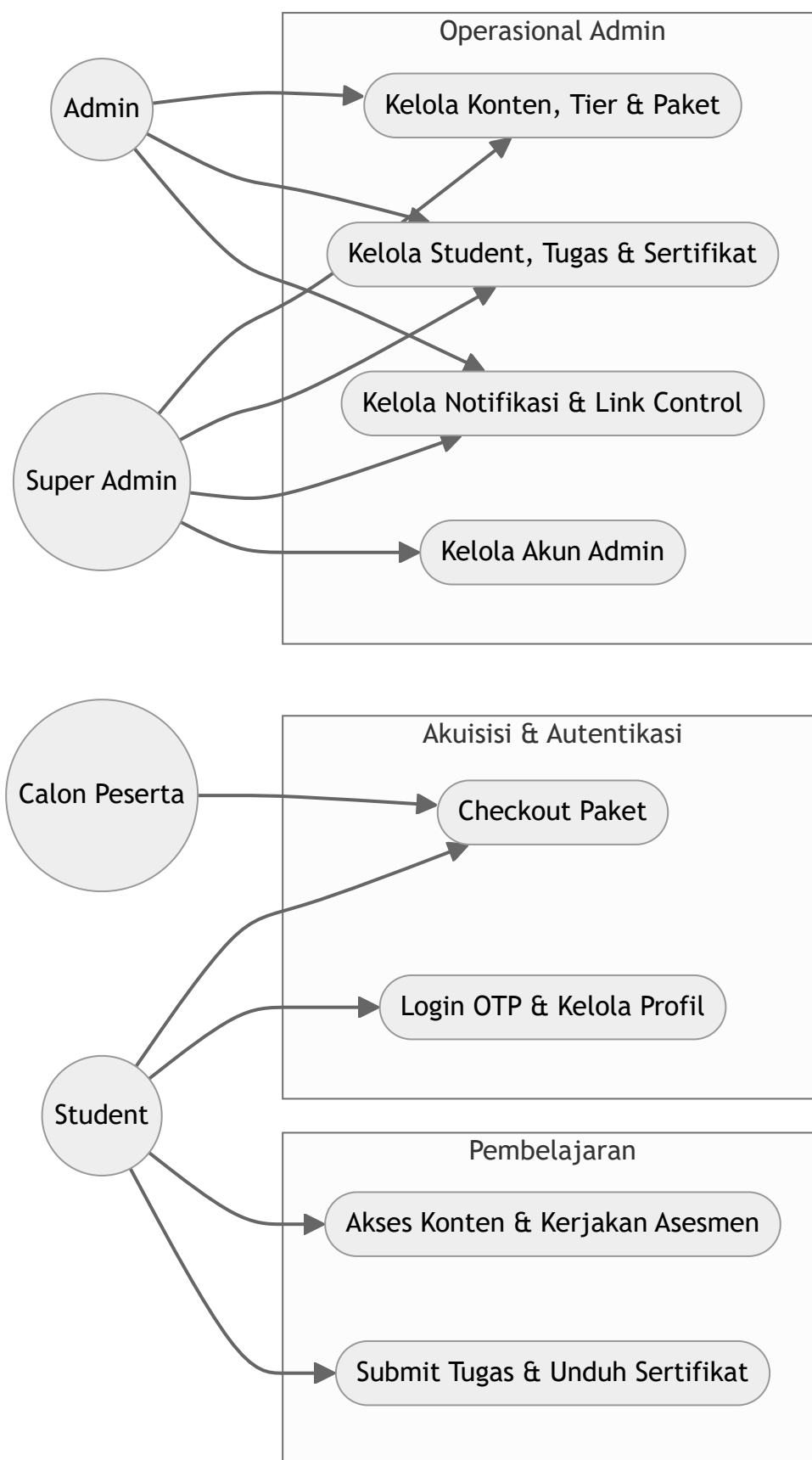


Gambar 2.2c — Aktivitas Tahap Tugas & Sertifikasi

2.3 Kelompok dan Karakteristik Pengguna

Kelompok Pengguna	Karakteristik & Hak Akses
-------------------	---------------------------

Super Admin	Hak akses penuh, termasuk seluruh kewenangan Admin biasa, ditambah kewenangan mengelola akun Admin lain (membuat, mengedit, menghapus akun admin). Tidak dapat menurunkan atau menghapus akun Super Admin lain/dirinya sendiri.
Admin	Staf operasional YogaFX yang mengelola konten pembelajaran, tier & paket, akun student, asesmen, sertifikat, dan notifikasi email. Tidak dapat membuat/mengedit/menghapus akun admin lain.
Student (Peserta)	Pengguna akhir yang telah menyelesaikan checkout dan onboarding. Mengakses konten belajar sesuai tier keanggotaannya, mengerjakan asesmen dan tugas, mengunduh sertifikat, serta dapat melakukan upgrade tier. Dibatasi lebih lanjut oleh status aktif (is_active) dan kelengkapan profil.
Calon Peserta (Lead/Publik)	Pengunjung anonim yang mengakses halaman publik/landing untuk melakukan pendaftaran minat (lead registration) dan checkout, sebelum menjadi Student resmi melalui proses onboarding. Belum memiliki akun sistem.
Pengguna Aplikasi Mobile	Student yang mengakses fitur setara (belajar, asesmen, tugas, sertifikat, profil) melalui aplikasi mobile terpisah, terautentikasi via token (Sanctum) dan OTP.



Gambar 2.3 — Diagram Use Case Ringkas per Kelompok Pengguna

2.4 Batasan Desain dan Implementasi

Sub-bab ini disusun berdasarkan pembacaan langsung terhadap kode sumber (migration, model, service, dan route aktual pada repository per 2026-07-27), bukan dari dokumen turunan, agar akurat terhadap perilaku sistem yang sesungguhnya berjalan.

- **Teknologi:** backend Laravel 13 (PHP ^8.3) dengan Inertia.js sebagai penghubung ke frontend React 18 (Vite, Tailwind CSS 4, komponen Radix UI/shadcn); basis data PostgreSQL 17; queue & cache berbasis database/file; deployment melalui Docker (PHP-CLI + PostgreSQL). Struktur skeleton mengikuti gaya Laravel 11+ (tidak ada app/Console/Kernel.php terpisah — seluruh jadwal cron didefinisikan langsung di routes/console.php).
- **Peran sistem terbatas:** hanya tiga nilai role yang didukung pada kolom users.role — super_admin, admin, student (konstanta User::ROLE_*); penambahan role baru berada di luar cakupan implementasi saat ini.
- **Status akun Student bersifat tiga-nilai**, bukan sekadar boolean aktif/nonaktif: kolom users.account_status bernilai available, inactive, atau suspended (konstanta User::ACCOUNT_STATUS_*), didampingi kolom boolean is_active sebagai penanda lawas yang tetap dipelihara. Terdapat pula penanda student_tag (normal/tester) — akun bertag tester dikecualikan dari pengecekan anti-kecurangan waktu tonton (lihat 3.2.3).
- **Alur pendaftaran:** tidak ada self-signup publik tanpa pembayaran (route register bawaan Laravel Breeze tidak diaktifkan) — akun Student hanya dibuat melalui alur checkout & onboarding, atau dibuat langsung oleh Admin manapun (bukan hanya Super Admin) dari menu Students.
- **Integrasi pembayaran:** seluruh transaksi diproses melalui **PayPal** (Checkout Orders API untuk bayar penuh, Subscriptions API untuk cicilan, serta Webhook untuk konfirmasi asinkron); tidak ada payment gateway lain yang diimplementasikan saat ini. Setiap paket (Package) memiliki salah satu dari tiga tipe pembayaran — paid, free, atau donation (konstanta Package::PAYMENT_TYPE_*) — dengan skema donasi memiliki kolom nominal minimum dan anjuran tersendiri.
- **Media pembelajaran:** video lesson wajib melalui **Bunny Stream** (kolom lesson_video_id); penyimpanan berkas lain melalui Bunny Storage/lokal, dengan batas ukuran unggah berbeda per jenis berkas — hasil verifikasi langsung terhadap app/Support/UploadConstraints.php: gambar umum/thumbnaill **10 MB**, **berkas ebook 500 MB** (jauh lebih besar dari asumsi umum 50–100 MB), lesson workbook **100 MB** (format PDF/DOC/DOCX), lesson audio **50 MB**, video tugas (assignment) **100 MB** (ekstensi mp4/mov/webm/avi/m4v). Validasi diterapkan baik di frontend maupun melalui rule max: pada FormRequest backend.
- **Skema cicilan (installment)** hanya berlaku untuk pembayaran awal paket (initial package) yang sudah ditautkan ke sebuah Access Tier; jumlah cicilan dibatasi **2–99** kali secara konfigurasi (Package::MIN/MAX_INSTALLMENT_COUNT), namun dibatasi maksimum **15** siklus penagihan pada sisi provider PayPal (MAX_PROVIDER_INSTALLMENT_COUNT); belum mendukung cicilan untuk upgrade tier maupun perpanjangan langganan umum di luar paket awal.
- **Akses konten** ditentukan oleh relasi many-to-many antara AccessTier dan konten (Module, Lesson, Ebook, Course) melalui tabel pivot; Student hanya memiliki tepat satu access_tier_id aktif pada satu waktu. *Catatan konsistensi skema:* khusus tabel courses, kolom FK langsung access_tier_id masih tersisa berdampingan dengan tabel pivot access_tier_course (tidak pernah dihapus saat migrasi many-to-many diterapkan) — ini adalah sisa teknis (schema drift) yang perlu diperhatikan pengembang lanjutan, berbeda dari Module/Lesson/Ebook yang FK langsungnya sudah dihapus sepenuhnya.

- **Tidak ada kelas Policy/Gate Laravel** (folder app/Policies kosong) — seluruh aturan otorisasi granular (mis. larangan admin biasa membuat admin lain, larangan menghapus sesama Super Admin) diimplementasikan sebagai pengecekan inline (`abort_unless/abort_if`) di controller dan otorisasi FormRequest, bukan melalui lapisan Policy terpisah.
- **Keterpeliharaan dokumentasi:** seluruh keputusan implementasi pada dokumen ini diverifikasi langsung terhadap migration, model, service, dan route aktual di repository per tanggal penyusunan dokumen.

3 Kebutuhan Rinci

3.1 Kebutuhan Antarmuka

3.1.1 Antarmuka Sistem

Detail berikut diverifikasi langsung dari routes/web.php, routes/api.php, routes/auth.php, routes/console.php, bootstrap/app.php, dan seluruh berkas app/Http/Middleware/.php.*

- **Basis data:** PostgreSQL 17, diakses melalui Eloquent ORM Laravel (77 berkas migration terverifikasi). Entitas utama meliputi users, access_tiers, packages, pending_registrations, onboarding_states, modules, lessons, ebooks, courses, assignments, assignment_submissions, assessments, questions, question_options, assessment_attempts, certificates, invoices, payment_activities (nama tabel final, entitas model bernama Payment), payment_subscriptions, payment_subscription_events, email_templates, email_logs, link_control_settings, dan personal_access_tokens (Sanctum). Lihat Bagian 4 untuk skema rinci.
- **Rute web** (routes/web.php): terbagi menjadi rute publik/tamu (lead registration, landing tier/paket, checkout dengan *signed URL*), grup Student (prefix implisit, middleware role:student + student.active + track.student.session), dan grup Admin (prefix /admin, middleware role:admin,super_admin). Terdapat juga rute wildcard /{packageSlug} di baris paling akhir sebagai fallback landing paket legacy — harus tetap didefinisikan terakhir agar tidak menelan rute lain.
- **Rute API mobile** (routes/api.php): dipasang sebagai grup api Laravel sehingga path sesungguhnya berawalan /api/mobile/v1/*. Endpoint publik (login, OTP, forgot/reset password) tidak berotentikasi; endpoint media (audio, workbook, ebook, sertifikat) dilindungi middleware mobile.signed (skema tanda tangan relatif kustom, bukan signed bawaan Laravel); endpoint data Student dilindungi auth:sanctum + mobile.student.
- **Middleware kustom** (alias terdaftar di bootstrap/app.php): role (EnsureUserHasRole — 403 bila role tidak cocok), student.active (EnsureStudentAccountIsActive — redirect ke halaman *inactive* bila akun Student tidak aktif), track.student.session (TrackStudentSessionActivity — mencatat sesi tanpa memblokir), mobile.signed (ValidateMobileRelativeSignature — menolak dengan JSON 403 bila tanda tangan tidak valid), mobile.student (EnsureMobileStudentAccess — menolak non-Student atau Student nonaktif, lalu mencatat sesi mobile).
- **Queue & Job Scheduler:** Laravel queue berbasis database untuk pengiriman email asinkron. routes/console.php hanya mendefinisikan **dua** jadwal cron: email-notifications:send-reminders (setiap 10 menit, closure inline memanggil EmailNotificationService::sendInactivityReminders()) dan installments:sync-overdue-status (setiap jam, kelas SyncOverdueInstallmentsCommand) — keduanya tanpa withoutOverlapping(). Perintah `yogafx:installment-recover-first-payment` (RecoverInstallmentFirstPaymentCommand) tersedia namun **tidak dijadwalkan** — hanya dijalankan manual oleh operator, dan diblokir di lingkungan produksi kecuali diberi flag `--force`.
- **Layanan pihak ketiga:** PayPal (Checkout Orders API & Subscriptions API + Webhook, dengan verifikasi tanda tangan webhook server-to-server) untuk pemrosesan pembayaran; Bunny.net Stream API untuk streaming video pembelajaran; Bunny.net Storage untuk penyimpanan berkas media terproteksi.

- **Format error API mobile:** seluruh exception (`AuthenticationException`, `ValidationException`, `HTTP exception`) pada rute berawalan `api/mobile/v1/*` dirender ulang melalui `MobileApiResponse::error()` menjadi amplop JSON yang konsisten, bukan format error default Laravel.

3.1.2 Antarmuka Pengguna

- Antarmuka web dibangun sebagai server-driven SPA menggunakan React + Inertia, dengan komponen UI konsisten berbasis Tailwind CSS dan Radix UI/shadcn.
- **Sisi Admin:** sidebar kiri collapsible, topbar berisi judul halaman dan menu pengguna; pola kerja *list* → *create/edit* → *kembali ke list* konsisten pada seluruh modul CRUD, dilengkapi pencarian, filter, dan pagination.
- **Sisi Student:** halaman Home bersifat content-first, menampilkan continue learning, ringkasan waktu akses, grid module, milestone tugas & sertifikat, serta bagian sumber daya ebook.
- Editor kaya (CKEditor 5); pemutaran video (Video.js/HLS dari Bunny Stream); pemotongan gambar profil (react-easy-crop).
- Pesan error dan validasi ditampilkan inline pada form, mengikuti mekanisme validasi Laravel yang diteruskan melalui Inertia.

3.1.3 Antarmuka Perangkat Keras

Tidak ada kebutuhan khusus terhadap perangkat keras tertentu. Sistem diakses melalui peramban web standar pada perangkat desktop/mobile, serta melalui aplikasi mobile pendamping pada perangkat Android/iOS yang berkomunikasi dengan backend melalui API mobile.

3.1.4 Antarmuka Komunikasi

- **HTTP/HTTPS:** seluruh komunikasi web dan API menggunakan protokol HTTP/HTTPS; `APP_FORCE_HTTPS` dapat diaktifkan untuk memaksa koneksi terenkripsi.
- **Email (SMTP):** untuk pengiriman kode OTP login, notifikasi transaksional, dan notifikasi siklus tagihan installment.
- **Webhook:** endpoint `POST /webhooks/paypal` (dikecualikan dari verifikasi CSRF) menerima notifikasi status pembayaran/subscription secara asinkron; setiap payload wajib lolos verifikasi tanda tangan PayPal sebelum diproses (lihat 3.3.3).
- **Signed URL:** tautan checkout, onboarding, dan reset password menggunakan mekanisme *signed URL* bawaan Laravel (middleware `signed`); akses media terproteksi pada API mobile menggunakan skema tanda tangan relatif kustom terpisah (middleware `mobile.signed`), berbeda mekanismenya dari signed URL web.
- **Format pesan:** REST API mobile menggunakan format JSON standar untuk request dan response.

3.2 Spesifikasi Kebutuhan Fungsional

3.2.1 Autentikasi & Kontrol Akses

3.2.1.1 Deskripsi dan Prioritas

Fungsi ini menangani proses masuk, verifikasi, dan pembatasan akses pengguna ke area privat sistem (Student/Admin). Berhubungan dengan fungsi utama Autentikasi & Otorisasi (Bagian 2.2). **Prioritas: Tinggi.** Rincian

berikut diverifikasi langsung dari `AuthenticatedSessionController`, `EmailOtpChallengeController`, `EmailOtpChallengeService`, model `AuthEmailOtpChallenge`, dan `User::postLoginRouteName()`.

3.2.1.2 Kebutuhan Fungsional

- Sistem harus membedakan akses berdasarkan role: `super_admin`, `admin`, `student`.
- Login web wajib dua tahap untuk **setiap** percobaan login yang berhasil melewati password (bukan hanya pada perangkat baru): (1) validasi email+password dengan pembatasan laju standar Laravel (maksimum 5 percobaan per kombinasi email+IP), lalu (2) verifikasi kode OTP 6 digit yang dikirim ke email terdaftar.
- Kode OTP disimpan hanya dalam bentuk hash (tidak pernah plaintext di basis data), berlaku selama **15 menit tetap** (tidak dapat dikonfigurasi), dan setiap permintaan OTP baru otomatis membatalkan OTP sebelumnya yang belum terpakai untuk konteks yang sama.
- Sesi pengguna baru benar-benar dibuat (`Auth::login()`) hanya setelah OTP diverifikasi berhasil — sesi awal yang terbentuk sesaat setelah password benar akan di-*logout* kembali oleh sistem sebelum tantangan OTP ditampilkan, sehingga pengguna belum dianggap terautentikasi penuh selama tahap OTP berlangsung.
- Sistem harus menyediakan fungsi logout, lupa password, dan reset password. Alur reset password memerlukan **dua faktor sekaligus**: tautan bertanda tangan (signed link) yang dikirim ke email *dan* kode OTP 6 digit yang harus diketik manual pada halaman yang sama — tautan reset berlaku sesuai konfigurasi `auth.passwords.users.expire` (default 60 menit).
- Area Admin hanya dapat diakses oleh role `admin/super_admin`; area Student hanya dapat diakses oleh Student dengan `account_status = available` (bukan sekadar flag boolean).
- Student yang profilnya belum lengkap harus diarahkan ke halaman pelengkapan profil. Kelengkapan profil dicek terhadap 15 kolom wajib: `first_name`, `last_name`, `email`, `whatsapp`, `country`, `birth_date`, `gender`, `practicing_yoga_for`, `yoga_sequence_experience`, `hours_per_week`, `current_fitness_level`, `flexibility_rating`, `motivation`, `why_yogafx`, `how_did_you_find_us`.
- Prioritas pengalihan pasca-login (dievaluasi berurutan): (1) jika Student dan akun tidak aktif → halaman *Student Inactive*; (2) jika Student dan profil belum lengkap → halaman lengkapi profil; (3) selain itu → dashboard sesuai role (Admin/Super Admin ke Admin Dashboard, Student ke Student Dashboard/Home).
- Sistem harus menolak login/akses apabila kredensial salah, OTP kedaluwarsa/salah, atau akun nonaktif, disertai pesan error yang jelas.

Catatan keterbatasan yang terverifikasi dari kode: tidak ditemukan mekanisme pembatasan percobaan gagal (retry limit) maupun penguncian pada tahap verifikasi OTP itu sendiri, dan tidak ada endpoint kirim-ulang (resend) OTP pada alur web login — endpoint resend OTP hanya tersedia pada API mobile (`/api/mobile/v1/auth/login/resend-otp`). Ini adalah kondisi aktual sistem saat dokumen ini disusun, dicatat sebagai referensi bagi pengembangan keamanan lanjutan.

3.2.1.3 Urutan Stimulus/Respon

1. Pengguna memasukkan email dan password → sistem memvalidasi kredensial (dengan rate limiting) dan role yang diizinkan.
2. Jika valid, sistem membuat tantangan OTP (menghapus tantangan lama yang belum terpakai), mengirimkan kode 6 digit ke email, meng-*logout* sesi sementara, dan menampilkan form input OTP melalui tautan bertoken.
3. Pengguna memasukkan kode OTP → sistem memvalidasi hash kode dan masa berlaku (15 menit).

4. Jika OTP valid, sistem benar-benar membuat sesi (`Auth::login()`) dan mengarahkan sesuai prioritas redirect di atas.
5. Jika kredensial atau OTP tidak valid/kedaluwarsa, sistem menampilkan pesan error dan tetap di halaman terkait.

3.2.2 Akuisisi Publik, Checkout & Onboarding

3.2.2.1 Deskripsi dan Prioritas

Mengelola perjalanan calon peserta mulai dari pendaftaran minat, pemilihan paket, pembayaran, hingga menjadi Student resmi. **Prioritas: Tinggi.** Rincian berikut diverifikasi dari CheckoutController, PaymentCheckoutService, PaymentFinalizerService, PayPalService, dan Services\Payments\PaymentSubscriptionService.

3.2.2.2 Kebutuhan Fungsional

- Pengunjung publik harus dapat mengisi lead registration/scoreboard, menghasilkan satu baris `pending_registration` (status awal created).
- Sistem harus menyediakan landing page publik per tier (`/online`, `/starter-kit`, `/masterclass`) dan per paket (`/p/{packageSlug}`, dengan fallback wildcard `/packageSlug`).
- Checkout harus mendukung dua skema, keduanya diakses lewat rute checkout bertanda tangan (signed URL): **bayar penuh** via PayPal Checkout Order (intent CAPTURE), dan **cicilan/installment** via PayPal Subscription.
- Pada skema **bayar penuh**: sistem membuat satu Invoice (`type=initial`, `payment_type=pay_full`) dan satu baris Payment (tabel `payment_activities`, status awal pending) dalam satu transaksi database; ID order PayPal disimpan sebagai `payment_reference`.
- Pada skema **cicilan**: sistem membuat Invoice (`payment_type=installment`) dan satu baris PaymentSubscription (status awal draft) — **tanpa** membuat baris Payment di muka; produk/plan PayPal dibuat sekali lalu di-cache per fingerprint konfigurasi paket agar tidak duplikat. Setelah calon peserta menyetujui langganan di PayPal, status berubah menjadi `approval_pending`; status active dan pembayaran pertama baru dikonfirmasi lewat webhook (Bagian 3.2.6), bukan langsung dari sisi klien.
- Proses finalisasi pembayaran sukses (PaymentFinalizerService) bersifat idempoten dan menggunakan penguncian baris (row locking) agar aman terhadap pemanggilan ganda (mis. webhook yang terkirim lebih dari sekali).
- Setelah pembayaran (penuh) berhasil, sistem otomatis membuat/menemukan akun User beserta OnboardingState (status awal `awaiting_enrollment`) dan melanjutkan ke onboarding: enrollment lalu signup, sebelum akun Student dianggap aktif sepenuhnya.
- Sistem harus menolak checkout dengan tautan tidak sah/kedaluwarsa (signed URL tidak valid) atau paket tidak aktif/tidak tertaut ke Access Tier manapun.

3.2.2.3 Urutan Stimulus/Respon

1. Pengunjung mengisi lead registration → sistem menyimpan `pending_registration`.
2. Pengunjung membuka tautan checkout bertanda tangan → sistem menampilkan ringkasan paket dan opsi pembayaran (penuh/cicilan sesuai konfigurasi paket).
3. Pengunjung menyelesaikan pembayaran melalui PayPal (capture order untuk bayar penuh, approval subscription untuk cicilan).

4. Sistem menerima konfirmasi (langsung untuk bayar penuh, atau via webhook untuk cicilan) → memperbarui status Invoice/Payment/PaymentSubscription secara idempoten.
5. Sistem mengarahkan ke form enrollment, lalu signup.
6. Setelah signup selesai, sistem membuat akun Student aktif dan mengirim email selamat datang beserta PDF invoice.

3.2.3 Manajemen Konten Pembelajaran & Pengalaman Belajar

3.2.3.1 Deskripsi dan Prioritas

Mencakup pengelolaan konten oleh Admin serta konsumsi konten oleh Student dengan aturan penguncian berurutan (sequential locking). **Prioritas: Tinggi.** Rincian berikut diverifikasi dari Student\LessonCatalogController (lessonAdvanceGate, isLessonFullyComplete, updateProgress) dan Student\ModuleCatalogController.

3.2.3.2 Kebutuhan Fungsional

- Admin harus dapat melakukan CRUD terhadap Module, Lesson, Assignment, Ebook, Course/Video Lecture, dan Dialog Content.
- Module, Lesson, dan Ebook memiliki relasi many-to-many terhadap Access Tier melalui tabel pivot; Course memiliki relasi many-to-many yang sama namun masih menyisakan kolom FK langsung access_tier_id (lihat catatan schema drift di Bagian 2.4).
- Lesson harus terhubung ke Module; video Lesson menggunakan lesson_video_id yang tervalidasi kesiapannya melalui Bunny Stream (HLS).
- Student hanya dapat melihat Lesson yang sesuai tier keanggotaannya dan status unlock saat ini.
- Sistem harus menyimpan watch_progress (persentase, 0–100) dan watch_time_seconds (akumulasi detik tonton nyata) per pasangan Student-Lesson. Lesson video dianggap selesai bila watch_progress ≥ 95 (nilai ambang persis, bukan pembulatan) **dan** akumulasi watch_time_seconds mencapai minimal durasi video dikurangi toleransi 2 detik (mekanisme anti-kecurangan agar progres tidak bisa di-skip paksa) — kecuali untuk akun berpenanda student_tag = tester yang dikecualikan dari pengecekan waktu tonton ini.
- Lesson berikutnya terkunci sampai **seluruh** prasyarat Lesson sebelumnya terpenuhi, dievaluasi berurutan: (1) bila Lesson memiliki berkas workbook, workbook tersebut harus sudah diunduh; (2) bila Lesson memiliki video, watch_progress ≥ 95 ; (3) bila Lesson memiliki Assessment berstatus live dan aktif, Student harus memiliki minimal satu AssessmentAttempt berstatus completed untuk asesmen tersebut (skor berapa pun dianggap memenuhi syarat — lihat catatan di 3.2.4). Lesson pertama pada urutan Module selalu terbuka tanpa syarat.
- Sistem harus memicu email otomatis saat modul/kursus diselesaikan dan saat workbook pertama kali dipicu/diunduh.

Catatan inkonsistensi implementasi yang terverifikasi dari kode: pengecekan syarat "workbook sudah diunduh" pada langkah (1) di atas ditemukan hanya diterapkan pada logika di LessonCatalogController; salinan logika yang sama pada ModuleCatalogController (dipakai untuk menampilkan status kunci pada halaman daftar/detail Module) tidak menyertakan pengecekan ini. Akibatnya, status terkunci/terbuka yang ditampilkan pada halaman Module berpotensi tidak 100% konsisten dengan halaman Lesson itu sendiri untuk kasus Lesson yang memiliki workbook. Dicatat sebagai temuan teknis untuk perbaikan lanjutan, bukan kebutuhan yang disengaja.

3.2.3.3 Urutan Stimulus/Respon

1. Admin membuat/mengedit Module beserta Lesson, menetapkan Access Tier yang berhak mengakses.
2. Student membuka Home/Modules → sistem menampilkan module sesuai tier dan status progres/unlock.
3. Student membuka Lesson dan memutar video → sistem mencatat watch_progress dan watch_time_seconds secara berkala.
4. Saat progres $\geq 95\%$, waktu tonton mencukupi, dan prasyarat lain terpenuhi, sistem menandai Lesson selesai dan membuka Lesson berikutnya.
5. Saat seluruh Lesson selesai, sistem mengirim email penyelesaian modul.

3.2.4 Asesmen (Assessment)

3.2.4.1 Deskripsi dan Prioritas

Menyediakan mekanisme evaluasi pemahaman Student melalui kuis/asesmen yang dapat dikaitkan dengan Lesson tertentu. **Prioritas: Sedang–Tinggi.** Rincian berikut diverifikasi dari Student\AssessmentController dan AssessmentResultsService.

3.2.4.2 Kebutuhan Fungsional

- Admin harus dapat membangun asesmen melalui builder: pertanyaan (11 tipe soal, mis. radio_buttons, multiple_choice_checkboxes, sliding_scale, numeric, open_text, dll.), opsi jawaban, jump logic, rentang hasil, dan pengaturan tampilan/branding (logo, margin, footer).
- Asesmen hanya dapat dibuka Student jika Lesson terkait memiliki asesmen aktif dan Student memenuhi aturan unlock Lesson (Bagian 3.2.3).
- Bila asesmen memiliki batas waktu (duration_minutes terisi), sistem menetapkan expires_at = waktu_mulai + duration_minutes saat percobaan dimulai. **Batas waktu ditegakkan secara lazy** (dicek ulang setiap kali Student membuka/menjawab), bukan melalui job latar belakang yang otomatis menutup percobaan pada detik kedaluwarsanya — percobaan yang sudah lewat waktu akan dipaksa berstatus expired pada interaksi berikutnya.
- Untuk pertanyaan berbasis opsi yang memiliki opsi bertanda is_correct, sistem harus memvalidasi dan menolak (menampilkan pesan error, tidak melanjutkan) pemilihan jawaban yang salah sebelum Student dapat lanjut ke pertanyaan berikutnya; pertanyaan numerik/teks bebas tidak memiliki gerbang kebenaran seperti ini.
- Logika lompat pertanyaan (jump logic) dievaluasi dengan prioritas: opsi jawaban terpilih yang memiliki aturan lompat sendiri diutamakan; jika tidak ada, gunakan aturan lompat pada pertanyaan itu sendiri; jika tidak ada juga, lanjut ke pertanyaan berikutnya berdasarkan urutan (sort_order).
- Sistem harus menyimpan percobaan (attempt), jawaban tiap pertanyaan, skor total, label hasil, dan ringkasan progres (skor terakhir, skor tertinggi, total percobaan) per Student per Assessment.
- Skor total dihitung dari akumulasi score_awarded tiap jawaban (nilai skor opsi terpilih untuk pertanyaan opsi dengan skor aktif, atau nilai numerik mentah untuk pertanyaan numerik; pertanyaan teks/terbuka selalu bernilai 0). Rentang hasil (AssessmentResultRange) dipilih berdasarkan kecocokan pertama $\text{min_score} \leq \text{total_skor} \leq \text{max_score}$ (kedua batas inklusif).
- Student harus dapat memulai atau melanjutkan (resume) pengerjaan asesmen yang belum selesai.

- Menyelesaikan sebuah percobaan asesmen — dengan skor berapa pun — menandai syarat "asesmen selesai" pada gerbang unlock Lesson (Bagian 3.2.3) sebagai terpenuhi; sistem **tidak** mensyaratkan skor/rentang hasil tertentu untuk membuka Lesson berikutnya.
- Admin harus dapat melakukan preview asesmen serta meninjau daftar dan detail hasil, termasuk menghapus percobaan tertentu.

3.2.4.3 Urutan Stimulus/Respon

1. Student membuka intro asesmen → sistem memvalidasi kelayakan akses.
2. Student memulai/melanjutkan → sistem menampilkan pertanyaan sesuai jump logic, sambil mengecek kedaluwarsa waktu bila berlaku.
3. Student mengirimkan jawaban tiap tahap → sistem memvalidasi (termasuk gerbang jawaban benar/salah bila berlaku), menyimpan jawaban dan progres.
4. Setelah selesai, sistem menghitung skor total, memilih rentang hasil yang cocok, menampilkan halaman hasil, dan menandai gerbang unlock Lesson terpenuhi.
5. Admin dapat membuka detail hasil per Student.

3.2.5 Tugas (Assignment) & Sertifikasi

3.2.5.1 Deskripsi dan Prioritas

Mengelola pengumpulan tugas video oleh Student pada tier relevan, peninjauannya oleh Admin, serta penerbitan sertifikat kelulusan. **Prioritas: Sedang–Tinggi** (saat ini aktif untuk tier online). Rincian berikut diverifikasi dari Student\AssignmentController, Admin\StudentProgressController::updateAssignment, CertificateEligibilityService, CertificateGeneratorService, dan config/certificates.php.

3.2.5.2 Kebutuhan Fungsional

- Student pada tier relevan harus dapat mengunggah video tugas (maks. 100 MB, ekstensi mp4/mov/webm/avi/m4v) untuk Assignment aktif yang terhubung ke Module.
- Pengunggahan ulang (resubmit) menggantikan video tugas sebelumnya (berkas lama dihapus dari penyimpanan Bunny) dan selalu mengembalikan status ke submitted, menghapus feedback dan tanggal review sebelumnya. **Tidak ada pembatasan berbasis status** pada resubmit — Student tetap dapat mengunggah ulang meski status sebelumnya sudah approved atau rejected.
- Admin harus dapat meninjau dan mengubah status ke salah satu dari lima nilai (submitted, under_review, pending_review, approved, rejected) beserta catatan feedback; tidak ada aturan mesin-status (state machine) yang membatasi transisi tertentu dalam kode saat ini.
- Admin harus dapat men-generate, membuat ulang (setiap generate ulang menambah nomor versi dan mengganti berkas lama), mengunduh, dan menghapus (soft delete) sertifikat. Penerbitan sertifikat bersifat **hanya oleh Admin** — tidak ada jalur otomatis di sisi Student.
- Kelayakan sertifikat (learning_eligible) ditentukan oleh **dua syarat** saja: (1) tier Student memiliki pemetaan jenis sertifikat yang tersedia pada config('certificates.tiers'), dan (2) seluruh Assignment yang relevan-sertifikat untuk Student tersebut memiliki submission berstatus approved (perbandingan jumlah lengkap, bukan persentase). **Tidak ada syarat penyelesaian Lesson/Module atau kelulusan Asesmen** pada algoritma kelayakan ini di kode saat ini.

- Terdapat dua jenis sertifikat tetap: `bikram_yoga_certificate` dan `yoga_alliance_certification`. Pemetaan tier→jenis dikonfigurasi statis: tier Starter Kit hanya berhak Bikram; tier Online dan Master Class berhak keduanya.
- Student hanya boleh mengunduh sertifikat miliknya sendiri.

3.2.5.3 Urutan Stimulus/Respon

1. Student mengunggah video tugas → sistem menyimpan submission berstatus submitted.
2. Admin meninjau dari menu Student Progress → memberi status approved/rejected beserta feedback → sistem mencatat reviewed_at/reviewed_by dan mengirim email notifikasi status tugas.
3. Sistem mengevaluasi kelayakan sertifikat setiap kali status Assignment relevan berubah menjadi approved.
4. Setelah seluruh Assignment relevan disetujui, Admin men-generate sertifikat sesuai jenis yang dipetakan ke tier Student.
5. Sistem mengirim email penerbitan sertifikat; Student mengunduh sertifikat.

3.2.6 Komersial & Penagihan Installment

3.2.6.1 Deskripsi dan Prioritas

Mengelola konfigurasi paket harga (bayar penuh, cicilan, donasi), Access Tier, serta siklus penagihan berkelanjutan.

Prioritas: Tinggi. Rincian berikut diverifikasi dari InstallmentPlanCalculator, PaymentSubscriptionService, PayPalSubscriptionService, InstallmentWebhookHandler, OverdueInstallmentService, SyncOverdueInstallmentsCommand, dan RecoverInstallmentFirstPaymentCommand.

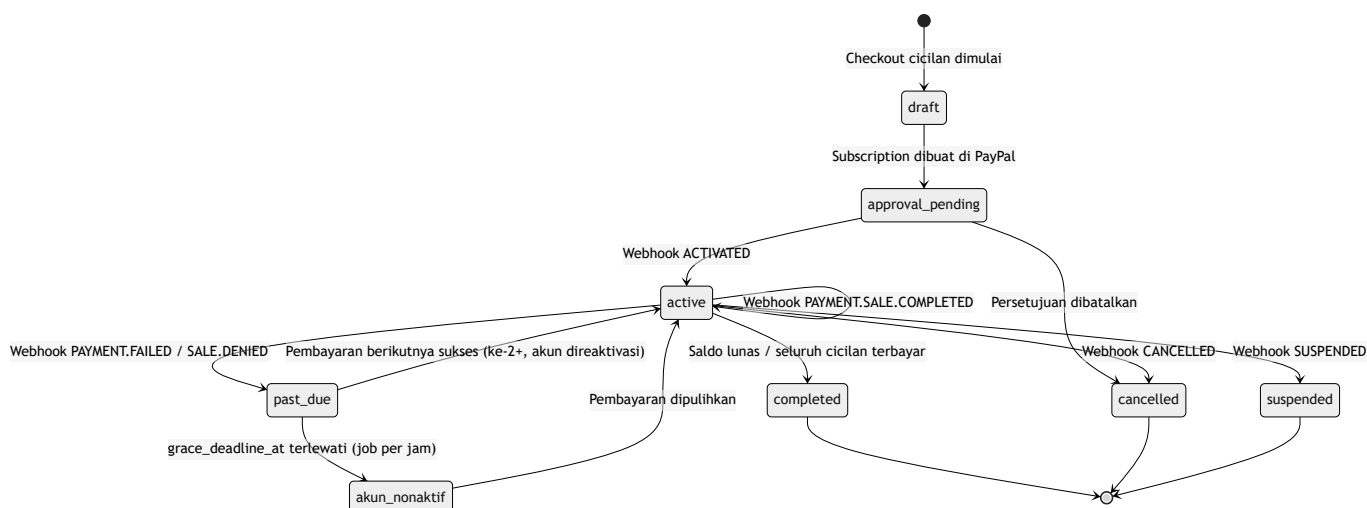
3.2.6.2 Kebutuhan Fungsional

- Admin harus dapat melakukan CRUD Package dan Access Tier. Package hanya tersedia untuk checkout publik apabila memiliki `access_tier_id` aktif.
- Sistem harus mendukung tiga tipe pembayaran paket: **berbayar (paid)**, **gratis (free)**, dan **donasi (donation)** dengan nominal donasi minimum/anjuran yang dapat dikonfigurasi per paket.
- Konfigurasi cicilan hanya boleh aktif bila Package sudah ditautkan ke Access Tier. Jumlah cicilan dihitung salah satu dari dua metode: **berbasis tanggal** (menghasilkan seluruh tanggal jatuh tempo bulanan hingga `installment_deadline_date` milik paket, dibatasi maksimum 15 siklus sisi PayPal) atau **berbasis jumlah** (jumlah cicilan tetap atau fleksibel sesuai konfigurasi, 2–99 kali). Hari penagihan pelanggan dapat dipilih tanggal **1 atau 15** setiap bulannya (bukan selalu tanggal 15).
- Pembagian nominal: $\text{cicilan reguler} = \text{floor}(\text{total} / \text{jumlah_cicilan})$; pembayaran pertama menyerap sisa pembulatan $(\text{total} - \text{reguler} \times (\text{jumlah} - 1))$ agar total tetap presisi.
- **Setiap tanggal jatuh tempo cicilan reguler memiliki tenggang waktu (grace period) tetap 3 hari** ($\text{grace_deadline_at} = \text{jatuh_tempo} + 3 \text{ hari}$) yang sudah ditetapkan sejak rencana cicilan dibuat — bukan hanya dihitung saat terjadi kegagalan pembayaran.
- Sistem harus mencatat setiap event webhook PayPal Subscription yang diterima (dengan deduplikasi berdasarkan ID event dari PayPal) untuk tipe: BILLING.SUBSCRIPTION.CREATED/ACTIVATED/CANCELLED/SUSPENDED/EXPIRED, BILLING.SUBSCRIPTION.PAYMENT.FAILED, PAYMENT.SALE.COMPLETED, dan PAYMENT.SALE.DENIED.

- Kegagalan pembayaran cicilan (event `PAYMENT.FAILED/PAYMENT.SALE.DENIED`) mengubah status subscription menjadi `past_due` dan menetapkan ulang `grace_deadline_at` ke jatuh-tempo-berjalan + 3 hari, lalu mengirim email kegagalan pembayaran.
- Sistem harus menjalankan job terjadwal **setiap jam** yang menonaktifkan akun Student (`is_active = false`) untuk setiap subscription berstatus `past_due` yang `grace_deadline_at`-nya sudah lewat, dan mengirim email peringatan tunggakan tepat satu kali per kejadian (dideduplikasi via log email).
- Pembayaran cicilan yang berhasil (event `PAYMENT.SALE.COMPLETED`) mencatat pembayaran baru, menambah hitungan cicilan terbayar, memajukan jatuh tempo berikutnya, dan menandai subscription `completed` begitu saldo lunas/jumlah cicilan tercapai. **Reaktivasi akun otomatis hanya terjadi mulai pembayaran cicilan kedua yang berhasil dan seterusnya** (bukan pada pembayaran pertama, karena akun belum pernah dinonaktifkan pada titik itu), dan hanya jika subscription belum berstatus final (`cancelled/suspended`).
- Sistem menyediakan perintah pemulihan manual (bukan terjadwal) untuk kasus pembayaran pertama yang gagal tercatat otomatis, dibatasi hanya dapat dijalankan operator di lingkungan non-produksi kecuali diberi flag paksa eksplisit.

3.2.6.3 Urutan Stimulus/Respon

1. Job terjadwal `installments:sync-overdue-status` berjalan tiap jam → memeriksa seluruh `payment_subscription` berstatus `past_due` dengan `grace_deadline_at` yang sudah lewat.
2. Untuk setiap subscription yang memenuhi kondisi tersebut, sistem menonaktifkan akun Student terkait dan mengirim email tunggakan (sekali per kejadian).
3. Saat webhook PayPal mengonfirmasi pembayaran berhasil, sistem mencatat event, membuat catatan pembayaran baru, memajukan jadwal, mengaktifkan kembali akun bila ini pembayaran kedua-dst dan subscription belum final, lalu mengirim email sukses.
4. Jika pembayaran gagal, sistem mencatat event, menandai subscription `past_due` dengan tenggang waktu baru, dan mengirim email gagal.



Gambar 3.1 — Diagram Status Payment Subscription pada Siklus Cicilan Installment

Catatan: status akun Student (`is_active`) bukan bagian dari mesin status `PaymentSubscription` di atas, melainkan efek samping yang dipicu oleh job per jam (nonaktif saat `past_due` dan tenggang 3 hari terlewati) dan oleh webhook pembayaran sukses ke-2 dan seterusnya (reaktivasi) — lihat Bagian 3.2.6.

3.2.7 Manajemen Peserta, Notifikasi Email & Link Control (Admin)

3.2.7.1 Deskripsi dan Prioritas

Fungsi operasional pendukung: pengelolaan akun Student/Admin, template & log notifikasi email, serta pengaturan tautan unduh aplikasi mobile. **Prioritas: Sedang.** Rincian berikut diverifikasi dari Admin\StudentController dan Admin\AdminAccountController beserta AdminAccountStoreRequest/AdminAccountUpdateRequest (tidak ada kelas Policy/Gate terpisah — seluruh otorisasi bersifat inline).

3.2.7.2 Kebutuhan Fungsional

- Admin harus dapat melihat, mencari, memfilter, dan paginasi daftar Student dan Admin.
- **Admin manapun** (bukan hanya Super Admin) berwenang membuat akun Student baru dari menu Students, dengan syarat minimal email (unik), password (min. 8 karakter, terkonfirmasi), dan Access Tier.
- Admin harus dapat mengubah account_status Student (available/inactive/suspended) — perubahan ini murni mengubah kolom status tanpa efek berantai ke data lain.
- Admin harus dapat mereset progres belajar Student secara penuh atau per lingkup (video, asesmen, lesson, module). Reset progres **menghapus permanen** baris terkait pada assessment_answers, assessment_attempts, assessment_progress, lesson_progress (atau mengosongkan kolom tonton untuk lingkup video saja), assignment_submissions (termasuk berkas video di Bunny Storage), student_module_visits, dan certificate_download_events — **tidak** menyentuh sertifikat, invoice/pembayaran, atau akun itu sendiri.
- Admin harus dapat menghapus akun Student secara permanen. Penghapusan adalah **hard delete** sesungguhnya (model User tidak menggunakan SoftDeletes) yang, dalam satu transaksi database, turut menghapus seluruh data terkait di ±15 tabel (termasuk sertifikat yang sedianya soft-delete dipaksa *force delete*, sesi, log OTP, permintaan ganti password, serta baris pending_registration/invoice/payment_subscription terkait) diikuti pembersihan berkas tersimpan (foto profil, sertifikat) di storage setelah transaksi berhasil.
- **Hanya Super Admin** yang berwenang membuat, mengedit, dan menghapus akun Admin (ditegakkan pada level otorisasi FormRequest, bukan hanya middleware rute). Aturan tambahan yang berlaku: Super Admin tidak dapat menghapus akun Super Admin manapun (termasuk dirinya sendiri, dan juga Super Admin lain, bukan cuma dirinya sendiri); endpoint edit akun Admin **tidak dapat** menaikkan role seorang Admin biasa menjadi Super Admin (percobaan promosi otomatis diturunkan kembali ke admin); dan seorang Admin yang mengedit profilnya sendiri tidak dapat mengubah role miliknya sendiri melalui form ini.
- Admin harus dapat membuat template email, mengunggah media, mengirim email uji, serta melihat log pengiriman.
- Sistem mendukung 15 tipe notifikasi otomatis aktif yang terdaftar di backend, mencakup siklus belajar, autentikasi, dan siklus cicilan installment.
- Admin harus dapat mengelola tautan global Google Play dan App Store beserta QR code otomatis.

3.2.7.3 Urutan Stimulus/Respon

1. Admin membuka menu Students → mencari/memfilter → membuka detail untuk edit/ubah status/reset progres/hapus akun.
2. Super Admin membuka menu Admin → membuat/mengedit/menghapus akun Admin biasa sesuai batasan otorisasi di atas.
3. Admin membuka menu Email → menyunting template → mengirim email uji → menyimpan template.

4. Sistem memicu notifikasi otomatis sesuai event → mencatat pada Email Logs.
5. Admin membuka Link Control → memperbarui tautan → sistem meregenerasi QR code.

3.3 Kebutuhan Non Fungsional

3.3.1 Kebutuhan akan Performansi

- Streaming video harus memanfaatkan CDN Bunny Stream (HLS) agar waktu buffering minimal.
- Pencatatan watch_progress harus dilakukan secara periodik/asinkron agar tidak mengganggu pemutaran video.
- Pengiriman email harus dijalankan melalui queue asinkron agar tidak memblokir respons permintaan pengguna.
- Daftar data besar pada panel Admin harus mendukung paginasi untuk menjaga waktu muat halaman tetap responsif.
- Job terjadwal harus selesai dalam rentang waktu antar-eksekusi tanpa terjadi tumpang tindih eksekusi.

3.3.2 Kebutuhan akan Keselamatan

- Kegagalan proses pembayaran tidak boleh menyebabkan aktivasi akun Student tanpa pembayaran tervalidasi melalui webhook resmi PayPal; proses finalisasi pembayaran bersifat idempoten dan menggunakan penguncian baris untuk mencegah duplikasi akibat webhook yang terkirim berulang.
- Penonaktifan otomatis akibat tunggakan harus disertai notifikasi email, dan dilakukan tepat satu kali per kejadian (dideduplikasi) agar tidak mengirim email berulang setiap kali job per jam berjalan.
- Penghapusan akun Student oleh Admin bersifat **hard delete permanen dan tidak dapat dibatalkan** (bukan soft delete) — mencakup penghapusan berantai manual (bukan cascade database) di ±15 tabel terkait dalam satu transaksi, sehingga harus disertai konfirmasi eksplisit pada antarmuka untuk mencegah penghapusan tidak disengaja.
- Reset progres belajar juga bersifat penghapusan permanen pada tabel-tabel terkait (bukan arsip/soft delete) sehingga memerlukan konfirmasi serupa.
- Unggahan berkas dibatasi ukurannya sesuai jenis (10–500 MB tergantung jenis, lihat Bagian 2.4) dan divalidasi baik di frontend maupun backend (rule max: dan pembatasan ekstensi/MIME pada FormRequest).

3.3.3 Kebutuhan akan Keamanan

- Login web wajib melalui verifikasi dua langkah untuk setiap sesi: password (dengan rate limiting standar Laravel, maksimum 5 percobaan per kombinasi email+IP) diikuti OTP email 6 digit yang disimpan dalam bentuk hash, tidak pernah plaintext.
- Password disimpan dengan hashing (cast hashed pada Eloquent, menggunakan algoritma bcrypt bawaan Laravel).
- Akses area Admin dan Student dibatasi melalui middleware berbasis role (role) serta status akun tiga-nilai (student.active, memeriksa account_status).
- Akses media terproteksi pada web menggunakan signed URL bawaan Laravel; pada API mobile menggunakan skema tanda tangan relatif kustom (mobile.signed) yang divalidasi setiap permintaan.
- API mobile diautentikasi menggunakan token Laravel Sanctum per pengguna (auth:sanctum), dengan pemeriksaan tambahan role Student dan status aktif akun pada setiap permintaan (mobile.student).

- Seluruh payload webhook PayPal wajib lolos verifikasi tanda tangan server-to-server (memanggil endpoint verifikasi resmi PayPal) sebelum diproses; permintaan tanpa konfigurasi webhook_id yang valid akan selalu ditolak (fail-closed), dan endpoint ini dikecualikan hanya dari verifikasi CSRF, bukan dari verifikasi tanda tangan.
- Sistem harus mencatat aktivitas sesi Student (user_sessions) serta aktivitas tidak wajar selama pemutaran lesson (lesson_irregular_activities), dengan pengecualian anti-cheat waktu tonton untuk akun bertag tester.
- Komunikasi produksi harus berjalan melalui HTTPS (APP_FORCE_HTTPS).
- Data pembayaran sensitif tidak disimpan langsung oleh sistem — seluruh pemrosesan didelegasikan ke PayPal; sistem hanya menyimpan referensi/ID transaksi.
- Tidak ditemukan pembatasan percobaan gagal (retry limit) pada tahap verifikasi OTP login web di kode saat ini — dicatat sebagai area yang perlu diperkuat untuk penguatan keamanan lanjutan.

3.3.4 Atribut Kualitas Perangkat Lunak Lainnya

- **Kemudahan pemeliharaan:** dokumentasi produk internal wajib selaras dengan migration, model, dan route aktual.
- **Konsistensi pola UI:** seluruh modul CRUD Admin mengikuti pola list → create/edit → redirect ke list, dengan dialog konfirmasi pada aksi hapus.
- **Skalabilitas konten:** relasi many-to-many Access Tier ↔ konten memungkinkan penambahan tier tanpa perubahan struktural besar.
- **Kegunaan:** sisi Student content-first (premium, tenang), sisi Admin task-first (efisien).
- **Keterujian:** test suite berbasis PHPUnit tersedia untuk memverifikasi perilaku backend.

3.3.5 Aturan Kebutuhan Operasional

- **Super Admin ↔ Manajemen Akun:** hanya Super Admin berhubungan dengan fungsi pengelolaan akun Admin (Bagian 3.2.7).
- **Admin ↔ Konten, Komersial, Peserta, Notifikasi:** Admin (dan Super Admin) berhubungan dengan Bagian 3.2.2 (monitoring), 3.2.3, 3.2.4, 3.2.5, 3.2.6, dan 3.2.7.
- **Student ↔ Pembelajaran & Sertifikasi:** Student berhubungan dengan Bagian 3.2.1, 3.2.3, 3.2.4, 3.2.5, dan sebagian 3.2.6 (upgrade tier).
- **Lead/Publik ↔ Akuisisi:** pengguna publik hanya berhubungan dengan Bagian 3.2.2 sebelum resmi menjadi Student.
- **Job Terjadwal ↔ Sistem:** proses pengingat dan sinkronisasi cicilan berjalan otomatis, namun hasilnya harus dapat ditelusuri Admin melalui menu Students dan Email Logs.

4 Kebutuhan Lain dan Model Analisis

Kebutuhan Data

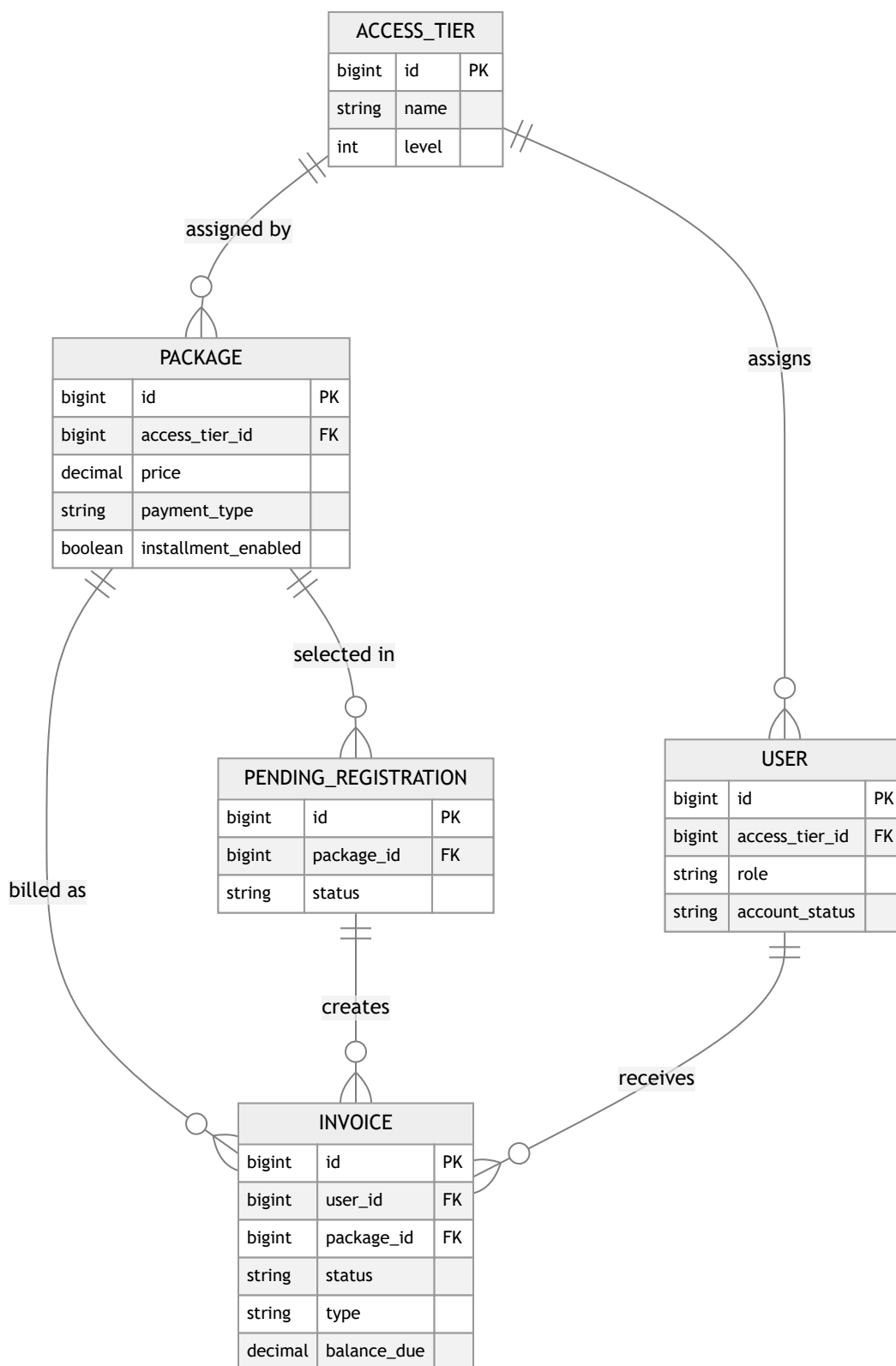
Basis data harus menyimpan riwayat lengkap transaksi (invoice, payment activity, payment subscription events) untuk keperluan audit dan rekonsiliasi keuangan. Data profil Student mencakup atribut spesifik domain yoga (practicing_yoga_for, yoga_sequence_experience, hours_per_week, current_fitness_level, flexibility_rating, motivation, dsb.) yang dikumpulkan pada tahap onboarding untuk personalisasi/analitik pembelajaran.

Kebutuhan Legal/Kepatuhan

Data pribadi peserta (email, WhatsApp, tanggal lahir, dsb.) harus dikelola sesuai prinsip kerahasiaan data pengguna; akses ke data ini dibatasi hanya untuk role Admin/Super Admin yang berwenang. Transaksi pembayaran didelegasikan sepenuhnya ke PayPal sebagai penyedia layanan pembayaran resmi, sehingga aspek kepatuhan PCI-DSS berada di luar tanggung jawab langsung sistem ini.

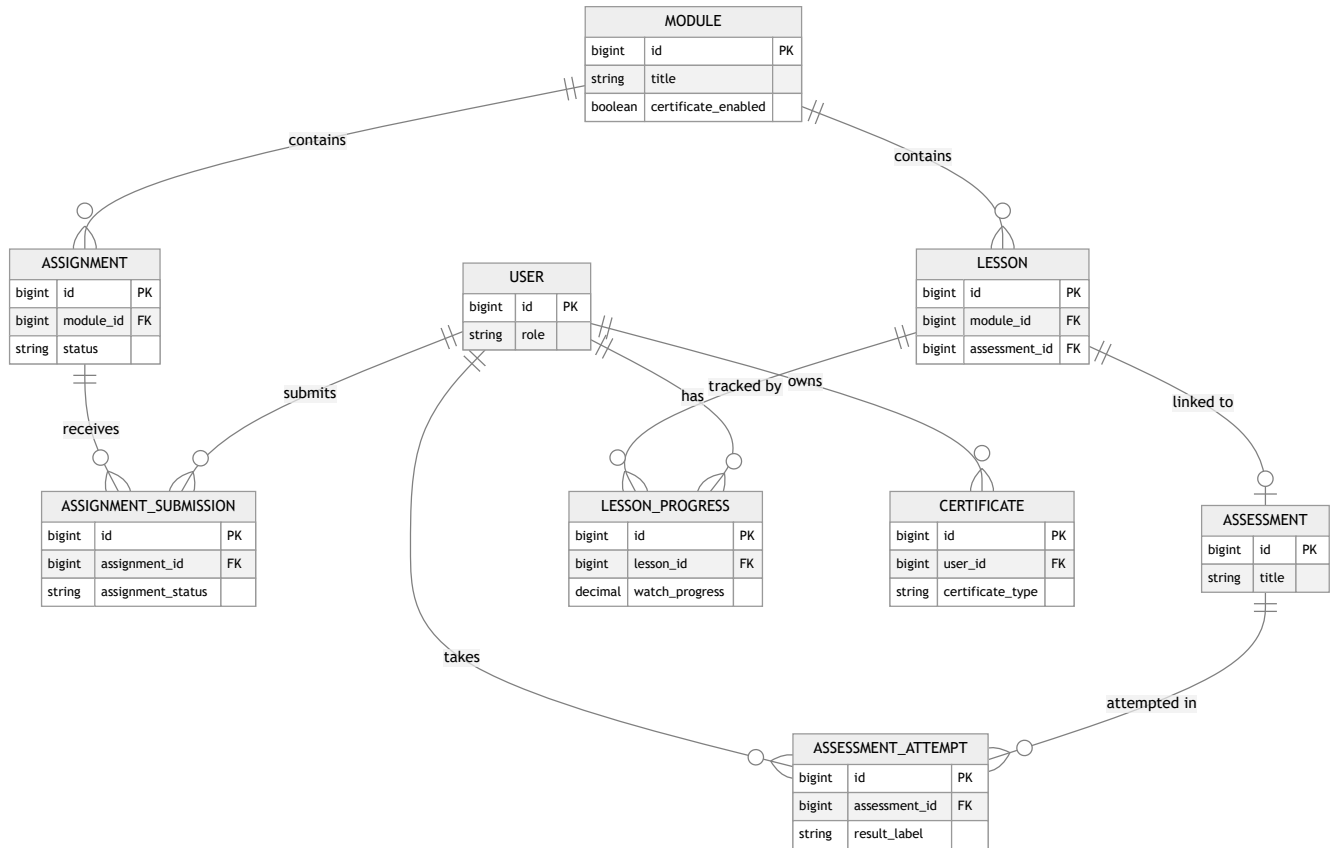
Model Analisis — Entity Relationship Diagram (ERD)

Diagram berikut menyajikan entitas inti dan relasi utama yang mendasari YogaFX LMS Web, disederhanakan (sebagian nama kolom dan kolom non-esensial diringkas) dari 77 berkas migration aktual pada repository, dipisah menjadi tiga bagian agar mudah dibaca: domain Akses & Komersial, domain Pembelajaran & Sertifikasi, dan domain Penagihan Installment.

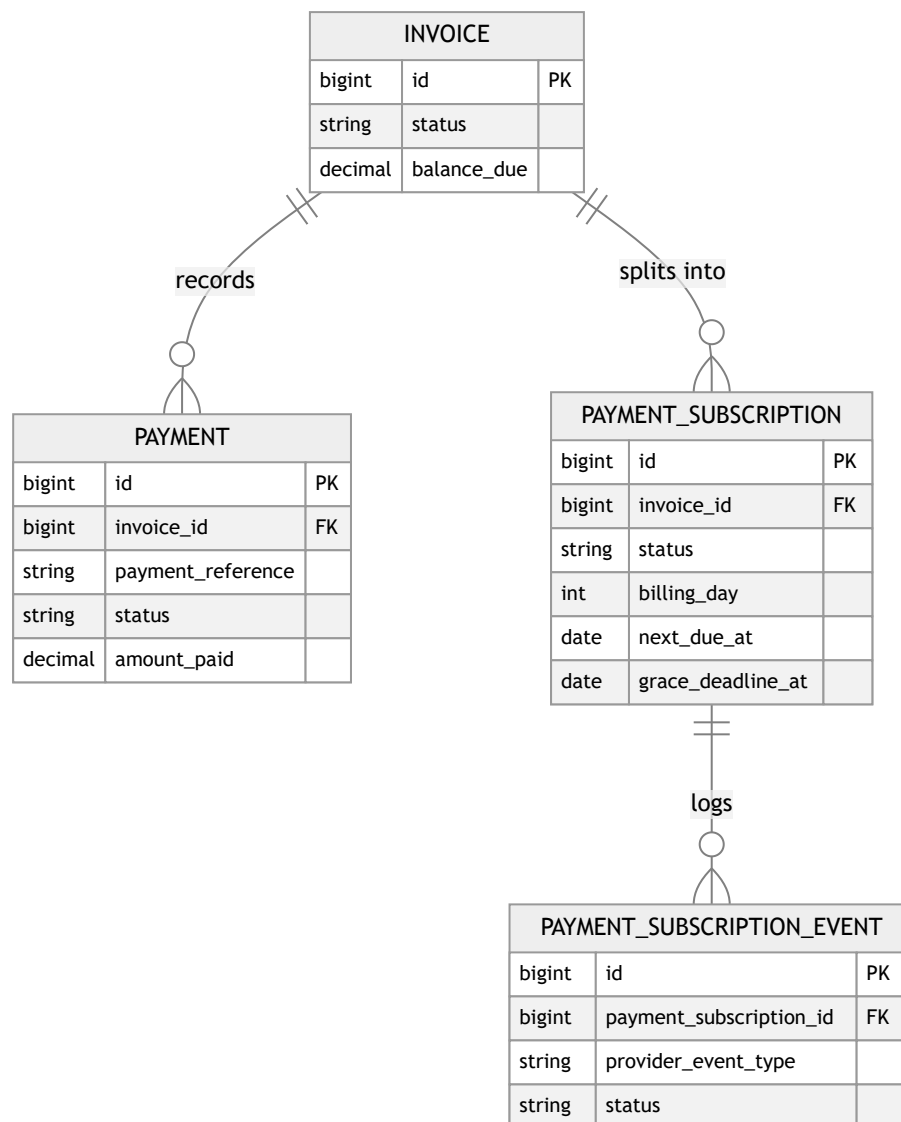


Gambar 4.1a — ERD (disederhanakan) — Domain Akses & Komersial

Catatan: kolom `users.account_status` bernilai `available/inactive/suspended` (bukan boolean sederhana); tabel `courses` secara skema masih menyimpan FK langsung ke `access_tiers` berdampingan dengan pivot many-to-many (lihat catatan schema drift, Bagian 2.4) dan tidak digambarkan di sini agar diagram tetap ringkas.



Gambar 4.1b — ERD (disederhanakan) — Domain Pembelajaran & Sertifikasi



Gambar 4.1c — ERD (disederhanakan) — Domain Penagihan Installment

Catatan: nama tabel basis data untuk entitas PAYMENT adalah payment_activities (nama sempat berganti payments secara singkat pada riwayat migrasi lalu dikembalikan); kelas model Eloquent-nya bernama Payment, dengan PaymentActivity sebagai sub-kelas alias tanpa kolom/tabel tambahan.

Ringkasan Alur Status Utama

- **Status Pendaftaran** (pending_registration.status): created → checkout_opened → payment_success → completed (onboarding selesai, Student aktif).
- **Status Pengumpulan Tugas** (assignment_submission.assignment_status): submitted → pending_review/under_review → approved atau rejected (dapat kembali ke submitted kapan pun via resubmit, tanpa batasan transisi — lihat Bagian 3.2.5).

- **Status Akun Student** (`users.account_status`): available secara default → dapat diubah Admin ke inactive/suspended, atau otomatis dinonaktifkan sistem saat subscription past_due dan tenggang 3 hari terlewati → dapat diaktifkan kembali oleh Admin atau otomatis mulai pembayaran cicilan sukses ke-2 dan seterusnya (lihat Gambar 3.1).
- **Status Payment Subscription** (`payment_subscriptions.status`): draft → approval_pending → active ↔ past_due → completed, atau cancelled/suspended sewaktu-waktu (lihat Gambar 3.1).

Referensi lebih lanjut mengenai struktur data lengkap dan alur pengguna tersedia pada dokumentasi produk internal repository: docs/03-erd.md (Entity Relationship Design lengkap), docs/02-user-flow.md (User Flow), dan docs/06-modular-implementation.md (Modular Implementation).